MUSIC AND
ACOUSTIC ENGINEERING

SELECTED TOPICS IN MUSIC AND ACOUSTIC ENGINEERING

Final Project

Giuliano Di Lorenzo
242712
Carmen Frieda Franci
259723
Nicola Antonio Magno
259037
Jorge Cuartero Galindo
273807



POLITECNICO
MILANO 1863

Contents

1 Introduction **2**

2 Data Extraction **2**

2.1 Stems . . . 2

2.2 Mixes . . . 2

2.3 Feature Extraction . . . 2

2.4 Label Mapping . . . 3

2.5 Final Dataset . . . 3

3 Model Design and Strategy **4**

3.1 First Stem Model – CQT-Based . . . 5

3.2 First Mix Model – CQT-Based (Multilabel) . . . 6

3.3 Stem Models – Reduced Architectures (Models 2, 3 and 4) . . . 8

3.3.1 Model 2 . . . 8

3.3.2 Model 3 . . . 8

3.3.3 Model 4 . . . 8

3.4 Mix Model – Lightweight Architecture (Model 2) . . . 9

3.5 Mel-spectrogram Models: Stem Model 5 and Mix Model 3 . . . 11

3.5.1 Stem – Model 5, with Mel-Spectrogram . . . 11

3.5.2 Mix – Model 3, with Mel-Spectrogram . . . 12

3.5.3 Mel vs. CQT: A Comparison . . . 13

3.6 Hybrid Model: Pretraining on Stem, Finetuning on Mix . . . 13

3.6.1 Comparison with Mix Model 3 . . . 14

3.7 Reduced Class Models: Model 6 for Stem and Model 4 for Mix . . . 15

3.7.1 Model 6 - Stem with Reduced Classes . . . 15

3.7.2 Model 4 - Mix with Reduced Classes . . . 16

3.7.3 Comparison with Previous Models Before Class Reduction . . . 17

3.8 Conclusion of the models . . . 17

4 Future directions **18**

4.1 Instrument-Genre Relationship Analysis . . . 18

4.2 Instrument Co-occurrence Analysis . . . 19

5 References **21**

1 Introduction

Automatic recognition of musical instruments is a central topic in the field of Music Information Retrieval (MIR), with applications ranging from automatic transcription and audio indexing to assisted music production.

Along this area, this project focuses on the development of a system capable of identifying musical instruments in audio recordings, addressing two main scenarios: classification of *stems* (individual audio tracks typically containing a single instrument) and detection of instruments in *polyphonic mixtures*, where multiple instruments are active simultaneously.

The reference dataset is **MedleyDB**, a collection of 122 professionally recorded multitrack songs, also with the associated stems, genre, temporal activations, and general information about the song. This allows both fine-grained analysis of individual instrument timbres and evaluation of classification models in more complex polyphonic settings.

The main objectives of the project include:

- Evaluating the **robustness** of instrument recognition systems for isolated and mixed situations.
- Study the **limits of the models** in terms of architecture and number of parameters.
- Evaluate **possible changes in the data** used to classify in order to look for the optimal configuration.

The proposed approach involves the extraction of features sensitive to timbral characteristics, and experimentation with various classification models. The overall goal is to assess the effectiveness of instrument classification systems in realistic scenarios, covering both monophonic and polyphonic contexts.

2 Data Extraction

To train and evaluate our instrument classification system, we constructed a custom dataset using the *MedleyDB* multitrack collection as original source. The dataset consists of 3-second audio excerpts extracted from both **stems** (isolated instrument tracks) and **mixes** (polyphonic recordings with multiple instruments).

2.1 Stems

For each stem, we used the activation annotations provided in the dataset to retain only the segments where the corresponding instrument is active. These 3-second segments were saved with a 50% overlap to increase the dataset size and improve generalization. All segments corresponding to the same instrument were grouped into a dedicated folder. We find 82 different instruments, with a very significant difference in the amount of samples for each, going from 0 (instruments whose samples are always less than 3 seconds) for the claps, to 6781 for the electric bass. Most of the instruments have less than 500 samples, so it's a very unbalanced data set for now.

2.2 Mixes

To extract segments from the mixture tracks, we analyzed the activation intervals of all instruments and selected only those 3-second regions where the set of active instruments remained constant throughout the entire segment duration. This guarantees that the instrument labels for each mixture segment are temporally coherent. Like the stems, these segments were also extracted with 50% overlap. As expected, because it's the same original data, this set it's also very unbalanced.

2.3 Feature Extraction

For each extracted audio segment (both from stems and mixes), we computed two time-frequency representations:

- **Mel-spectrogram** with 96 frequency bins: a time-frequency representation where the frequency axis is scaled according to the mel scale, which approximates the human auditory system's perception of pitch. It emphasizes perceptually relevant frequency bands, making it well-suited for timbre-related tasks.

- **Constant-Q Transform (CQT)** with 96 frequency bins: a time-frequency representation where each frequency bin has a constant Q-factor (ratio of center frequency to bandwidth). This results in logarithmically spaced frequency bins, similar to musical notes, making CQT particularly effective in capturing harmonic content and pitch-related features.

Each representation results in a matrix of size 96×130, corresponding to the 3-second duration and chosen hop size. These matrices were stored for later use in model training.

2.4 Label Mapping

The original MedleyDB dataset includes 82 distinct instrument labels. To reduce class imbalance and improve classification performance, we grouped several related instruments into broader categories. Instruments not included in our mapping were discarded.

The final classification problem consists initially of 18 classes, as summarized in the mapping below:

- **Individual classes**:
  - `electric_bass`
  - `drum_set`
  - `piano`
  - `synthesizer`
  - `clean_electric_guitar`
  - `distorted_electric_guitar`
  - `acoustic_guitar`
  - `violin`
  - `female_singer`
  - `vocalists`
- **Grouped classes**:
  - `male_vocals`: `male_singer`, `male_speaker`, `male_rapper`
  - `other_bowed_strings`: `viola`, `cello`, `double_bass`, and orchestral string sections
  - `woodwinds`: `flute`, `clarinet`, `saxophones`, `oboe`, etc.
  - `brass`: `trumpet`, `trombone`, `french_horn`, etc.
  - `other_percussion`: `auxiliary percussion instruments`, `timpani`, `tabla`, `shaker`, etc.
  - `other_plucked_strings`: `banjo`, `harp`, `mandolin`, `guzheng`, etc.
  - `reed_instruments`: `accordion`, `harmonica`, `melodica`
  - `fx_and_processed`: `scratches`, `processed_effects`

This organization not only tries to reduce the complexity of the classification, but also reduce the unbalance in the different classes, even though they will still be unbalanced.

2.5 Final Dataset

Once all the mel-spectrogram and CQT matrices were extracted and labeled, we stored them in various NumPy array files (`.npy`) to be used during model training. The final dataset consists of:

- **68,879** stem-based audio segments, each labeled with a single instrument class
- **9,154** mixture-based audio segments, potentially labeled with multiple instrument classes

Figures 1 and 2 show the distribution of instrument classes in the dataset for stem-based and mix-based segments, respectively. These histograms highlight the class imbalance, which motivated our decision to reduce the number of classes from 82 to 18 through grouping and filtering.

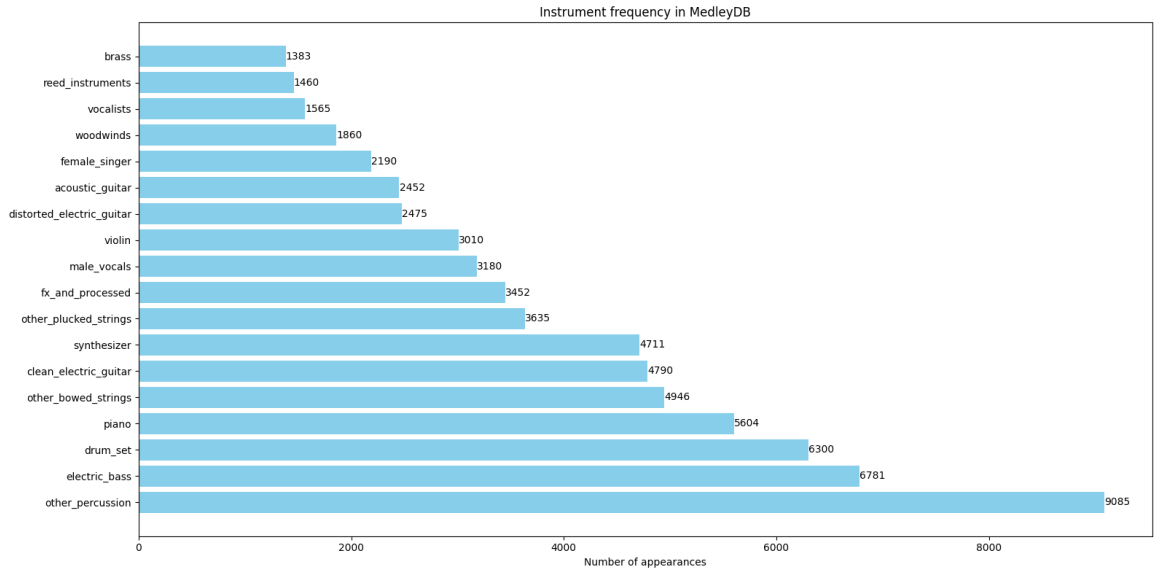


Figure 1: Class distribution for stem-based audio segments

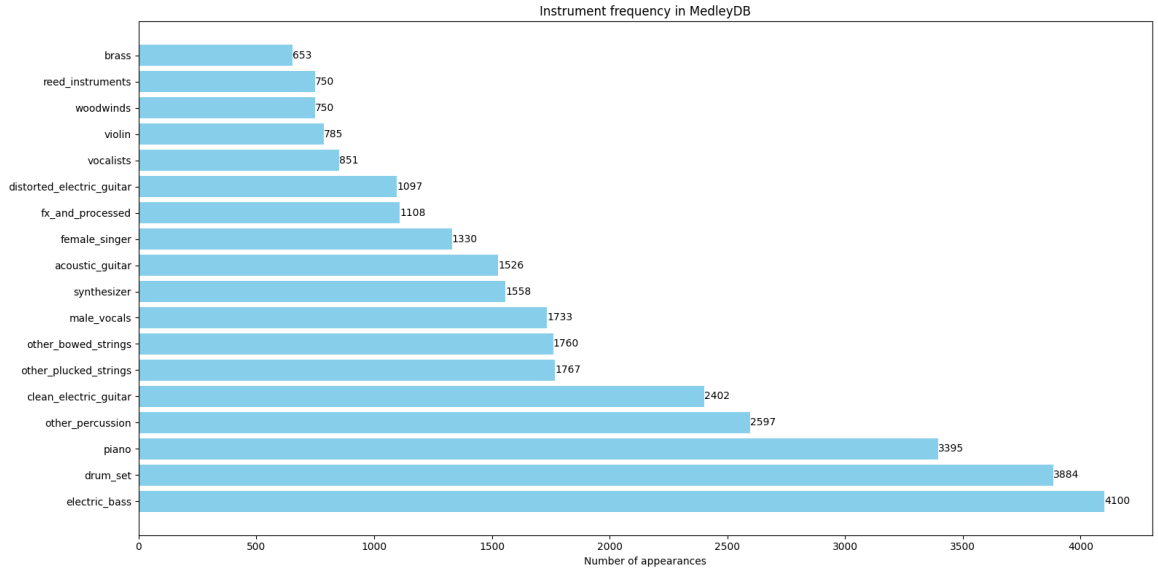


Figure 2: Class distribution for mix-based audio segments

We can appreciate that, even being the data still unbalanced, a reduction of the rate between the mean value and the variance of the distribution occurs. Considering stems as an example, we first had a sigma that was 1.7 times the average, but after the mapping, this is reduced to 0.6.

3 Model Design and Strategy

Several models were developed to address the instrument classification task, varying according to the dataset size and the type of input—whether isolated stems, polyphonic mixes, or a combination of both. We initially focused on training models exclusively on stems, as these provide cleaner, monophonic data that is easier to label and analyze. Based on the results obtained in this simpler context, we identified suitable architectures that could then be adapted to the more complex mix-based setting.

To ensure manageable training times and reduce class imbalance, we progressively simplified both the model architectures and the number of output classes. This strategy was applied not only to the stem and mix models individually, but also to a third category of models trained on both stems and mixes simultaneously, which we refer to as *Hybrid* models. These aim to generalize better by combining the clarity of isolated sources with the richness and complexity of real-world mixtures.

For each of the three settings—stems, mixes, and hybrid—we experimented with both mel-spectrogram and CQT representations, in order to evaluate the impact of the chosen time-frequency features on classification performance.

3.1 First Stem Model – CQT-Based

For the first model, we focused on stem data and used the Constant-Q Transform (CQT) as input representation, because due to its properties we intuitively thought it could be more accurate. All inputs were normalized using the global mean and standard deviation computed from the training set. This preprocessing step ensured stable convergence and balanced feature scaling.

The model architecture consisted of a convolutional neural network with three convolutional blocks, each followed by batch normalization and max pooling. These were followed by a fully connected layer with dropout, and a final softmax output layer. A summary of the architecture is shown below:

Layer	**Configuration**
Input	$96 \times 130 \times 1$ (CQT matrix)
Conv2D + BN + MaxPool	32 filters, 3×3, ReLU
Conv2D + BN + MaxPool	64 filters, 3×3, ReLU
Conv2D + BN + MaxPool	128 filters, 3×3, ReLU
Flatten	–
Dense + Dropout	128 units, ReLU + Dropout(0.3)
Output	Softmax over 18 classes

Table 1: Model architecture used for stem classification

Despite being relatively deep and parameter-heavy, this model achieved excellent performance without overfitting, albeit with a longer training time.

Figure 3 shows the training and validation accuracy and loss curves. The model converged rapidly, although the validation curve displayed some fluctuations due to the high complexity and size of the dataset.

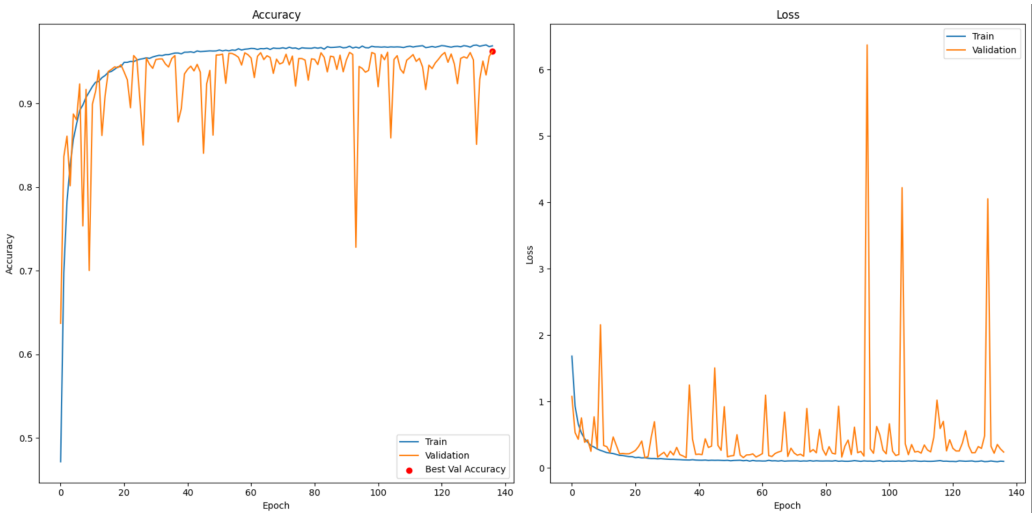


Figure 3: Training and validation accuracy/loss for the stem CQT model

The confusion matrices in Figure 4 confirm that most classes were correctly identified with very few misclassifications, even in the test set.

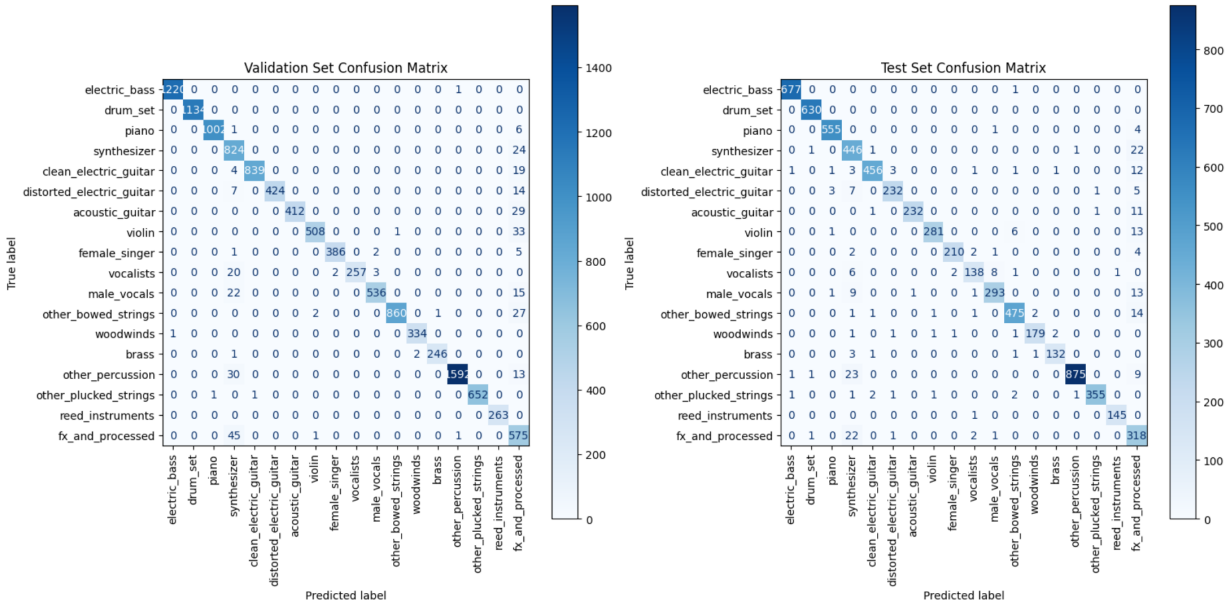


Figure 4: Confusion matrices on validation and test sets for the stem model

Classification reports further confirm the model's robustness. On the validation set, it achieved an overall accuracy of 97%, with macro-averaged precision and recall both around 98%. Even on the test set, which contained unseen data, the model maintained a strong accuracy of 96%. While most classes were predicted with nearly perfect scores, some drops were observed in classes such as `fx_and_processed` and `synthesizer`.

3.2 First Mix Model – CQT-Based (Multilabel)

For the mix dataset, we designed a separate model to account for the inherent polyphonic nature of the data, where multiple instruments may be active at the same time. As a result, the classification task becomes *multilabel*, unlike the stem case which was strictly single-label. This change also affects the architecture and evaluation metrics: the model uses a sigmoid output activation instead of softmax, and binary cross-entropy is used as the loss function. Consequently, the performance is evaluated in terms of binary accuracy.

The model is a 2D convolutional neural network (CNN), similar in spirit to the one used for stems, but adapted for multilabel learning. It consists of three convolutional layers followed by max pooling, global average pooling to reduce dimensionality, and two dense layers. A summary of the architecture is presented below:

Layer	**Configuration**
Input	$96 \times 130 \times 1$ (CQT matrix)
Conv2D + MaxPool	32 filters, 3×3, ReLU
Conv2D + MaxPool	64 filters, 3×3, ReLU
Conv2D	128 filters, 3×3, ReLU
GlobalAveragePooling2D	–
Dense	64 units, ReLU
Output	Sigmoid over 18 classes (multilabel)

Table 2: Architecture of the CNN model used for mix classification

As in the stem model, the input features were normalized using global statistics from the training set. The CQT representation was again chosen for its ability to capture harmonic content across instruments in polyphonic settings.

Figure 5 shows the training and validation binary accuracy and loss. The model converges smoothly with a final validation accuracy above 95%, and no evident signs of overfitting.

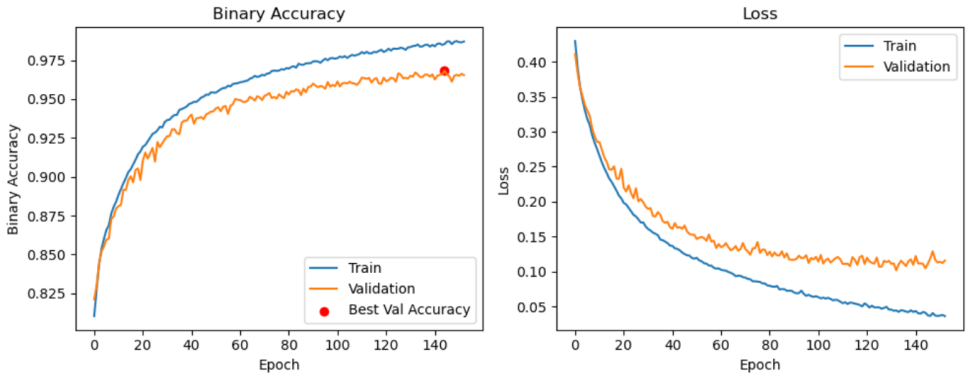


Figure 5: Binary accuracy and loss curves for the mix model (CQT-based)

The multilabel confusion matrices in Figure 6 show strong results across all 18 classes, with particularly high true positive rates for classes such as `electric_bass`, `drum_set`, `piano`, and `woodwinds`. Some lower recall values appear in classes like `male_vocals`, `fx_and_processed`, and `vocalists`, which could be due to fewer training samples or because they are very similar acoustically to other classes.

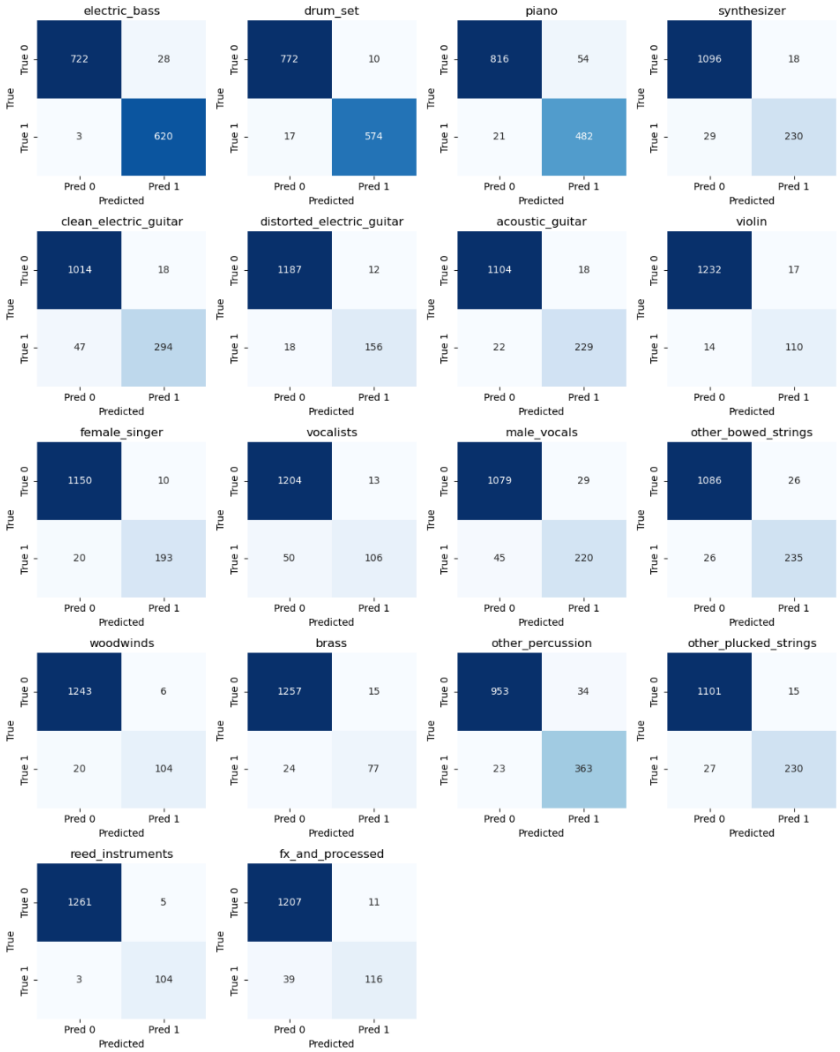


Figure 6: Binary confusion matrices for each class (mix model)

The classification report confirms these trends: on the validation set, the model achieved a micro-averaged F1-score of 0.92 and a macro average of 0.90. On the test set, performance remained consistently strong with a macro average F1 of 0.89 and micro F1 of 0.91. These results validate the model's ability to handle complex mixtures and detect overlapping sources reliably.

Point out that the value of recall it's a little bit smaller (87%). This can be concerning depending on our purpose with this model, because what we see it's that the model it's very good capturing when the model it's not playing, but it's not as good when discerning if an instrument it's not playing. This it's a direct consequence of the unbalance between the amount of silence and playing samples for a class, but it is something unavoidable considering not all instruments can be playing all the time. Still, the result it's not bad, but this unbalance can be a problem with future models.

Overall, this CNN-based multilabel model demonstrates solid generalization and robustness in the polyphonic context of musical mixes, proving that convolutional architectures can be adapted effectively to both single-label and multilabel instrument classification tasks.

3.3 Stem Models – Reduced Architectures (Models 2, 3 and 4)

After the first stem model, which was computationally heavy but highly performant, we developed three progressively lighter versions in order to evaluate the trade-off between model size and classification accuracy.

3.3.1 Model 2

The second model reduced the number of filters in each convolutional layer to half and decreased the size of the dense layer from 128 to 64 neurons. Despite this reduction, the architecture preserved the three convolutional blocks and used `Flatten` followed by a dense output layer.

This model performed remarkably well: accuracy on the validation set reached 96%, and the classification report showed strong results across most classes. Notably, precision and recall remained above 0.90 for the majority of instruments. A few exceptions, such as `vocalists` and `fx_and_processed`, exhibited slightly lower performance due to their complexity and data imbalance. These classes were already performing worse before.

3.3.2 Model 3

In the third model, we further reduced the architecture by shrinking the final dense layer to only 32 units. This change decreased the number of parameters and made the model faster to train.

Surprisingly, the performance remained close to model 2. The classification report indicated similar precision and recall for most classes, although a few rare categories showed slight performance degradation. In particular, `vocalists`, `distorted_electric_guitar`, and `male_vocals` had marginally lower F1-scores.

Despite the lighter architecture, this model maintained 95% accuracy on both validation and test sets, confirming the robustness of convolutional approaches even with fewer parameters.

Comparison (Model 2 vs. Model 3): Model 3 achieved nearly identical accuracy with fewer dense parameters, making it a more efficient option for deployment. However, model 2 showed slightly more stability and higher recall in some critical classes such as `male_vocals` and `fx_and_processed`.

3.3.3 Model 4

The fourth and final model aimed at maximizing efficiency by further simplifying the architecture. The third convolutional block was removed entirely, and the `Flatten` layer was replaced with a `GlobalAveragePooling2D` layer, drastically reducing the number of trainable parameters. This model represents a lightweight alternative suited for real-time or resource-constrained scenarios.

As expected, performance decreased compared to the previous models. However, it still reached a solid 91% accuracy on the test set. The classification report highlighted some challenges in detecting classes like `vocalists`, `distorted_electric_guitar`, and `fx_and_processed`, but retained high precision and recall for core classes such as `electric_bass`, `drum_set`, and `other_percussion`.

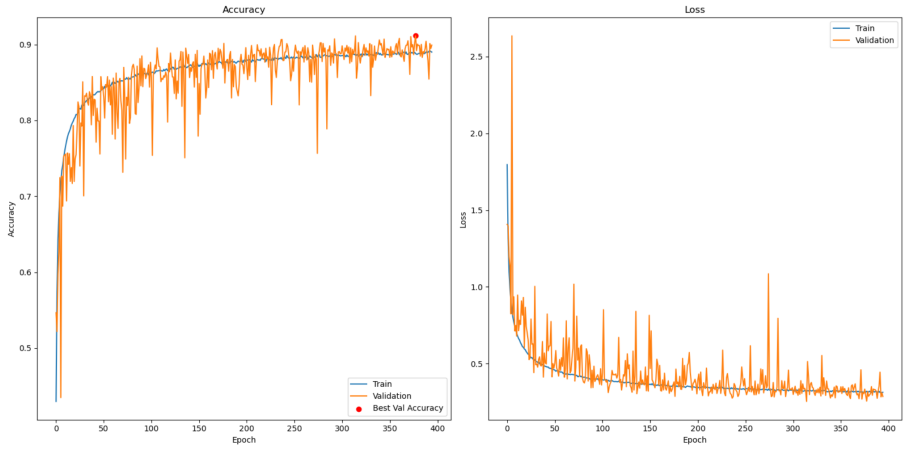


Figure 7: Accuracy and loss curves for stem model 4 (lightweight)

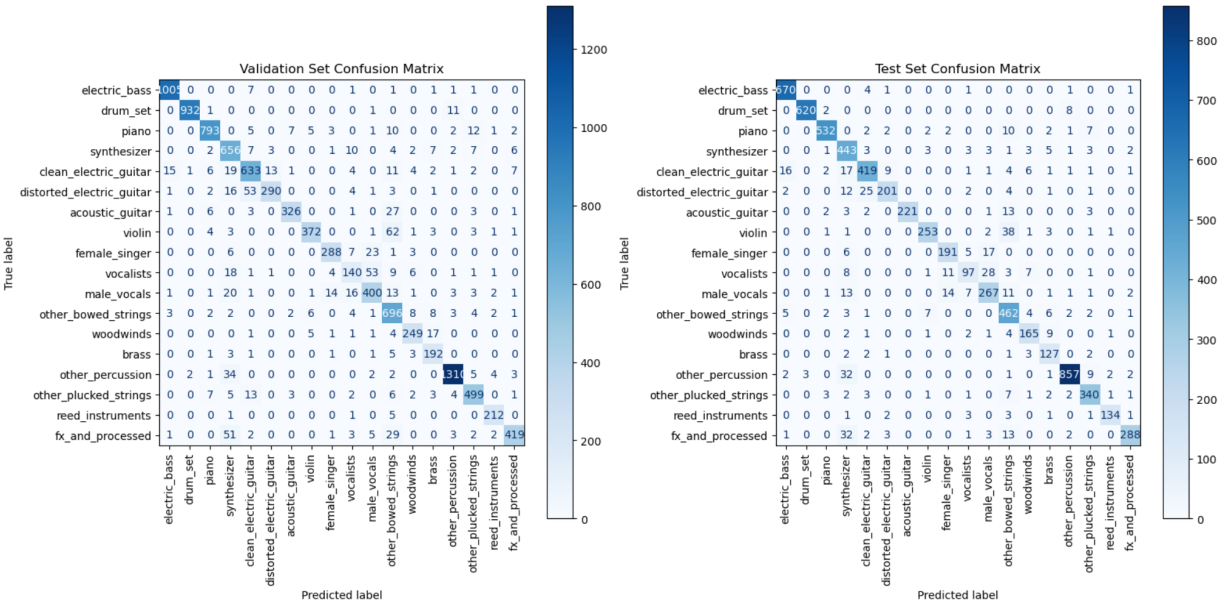


Figure 8: Confusion matrices for validation and test sets (model 4)

Overall, model 4 demonstrates that a compact convolutional network with global pooling can still provide decent classification accuracy, making it an attractive solution when computational efficiency is critical.

3.4 Mix Model – Lightweight Architecture (Model 2)

After observing that the fourth stem model—despite its simplicity—still achieved solid results, we decided to adopt the same architecture as a baseline for the second mix model. The goal was to develop a lightweight yet effective model that could be deployed efficiently while maintaining good performance on the multilabel classification task.

This architecture uses only two convolutional layers followed by a `GlobalAveragePooling2D` layer instead of a `Flatten`, significantly reducing the number of trainable parameters. The model was trained on mix data using the same CQT-based representation and binary cross-entropy loss, given the multilabel nature of the task.

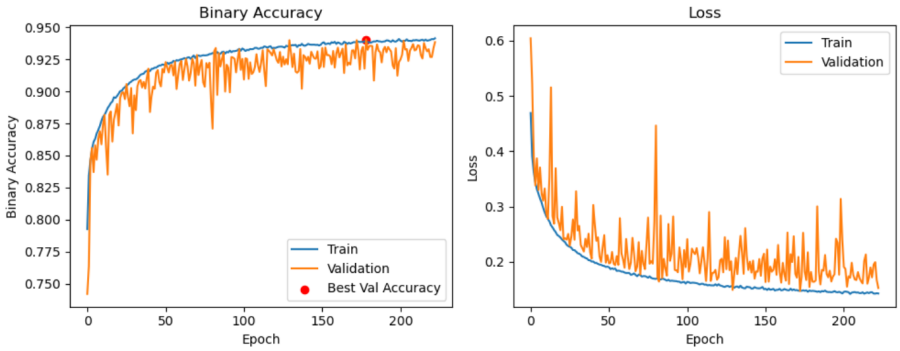


Figure 9: Binary accuracy and loss curves for mix model 2 (lightweight)

The accuracy and loss curves in Figure 9 show a stable training process, with validation accuracy reaching around 93% and no significant overfitting. While slightly lower than the first, more complex mix model, this performance is still very competitive.

Figure 10 displays the binary confusion matrices per class. Since our primary interest lies in identifying when an instrument is active, we focus on the true positive detection (i.e., the bottom-right cell of each matrix). Therefore, recall becomes a key metric.

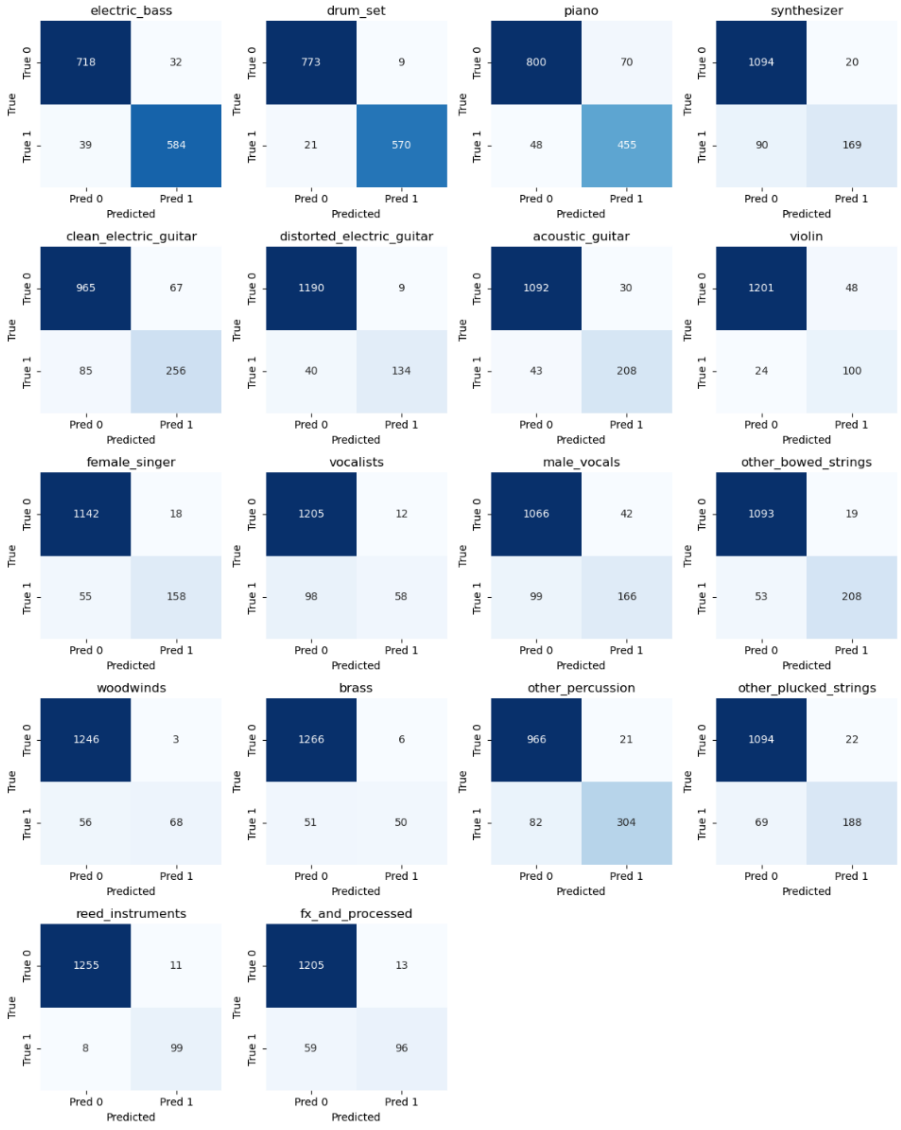


Figure 10: Binary confusion matrices per class for mix model 2

The classification report reflects this focus. While common instruments such as `drum_set`, `electric_bass`, and `piano` achieved very high recall values, several others—especially vocals and less frequent instruments like `fx_and_processed` or `male_vocals`—showed lower recall. This suggests that while the model performs well on dominant or clearer instruments, it struggles more with ambiguous or overlapping sources.

Overall, the model still manages to reach a micro-averaged F1-score of 0.84 and a macro average of 0.79. These results confirm that this compact architecture can be used as a viable solution when computational efficiency is required, although some loss in recall and generalization for minor classes is expected.

This fact motivate us to try to look for other ways to improve our results, in order to be able to keep this light architecture.

3.5 Mel-spectrogram Models: Stem Model 5 and Mix Model 3

In the previous experiments, we relied on the Constant-Q Transform (CQT) representation. For this set of models, we kept the same architecture as in Stem Model 4 and Mix Model 2, but switched to using the **mel-spectrogram** as input. The mel-spectrogram provides a perceptually motivated frequency scale, mimicking the resolution of human hearing by allocating more frequency bins to lower frequencies. This representation is widely used in audio classification tasks due to its ability to capture timbral information in a compact and effective way.

3.5.1 Stem – Model 5, with Mel-Spectrogram

Stem Model 5 uses the same lightweight architecture as Model 4 (two convolutional blocks followed by global average pooling and two dense layers), but is trained on mel-spectrograms instead of CQTs.

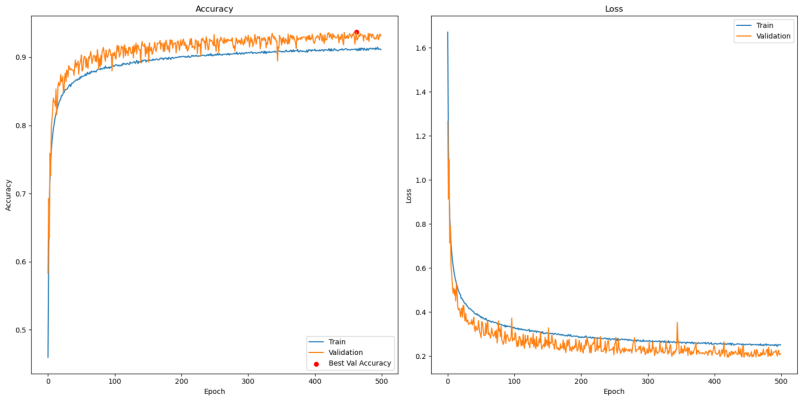


Figure 11: Accuracy and loss curves for stem model 5 (mel-spectrogram)

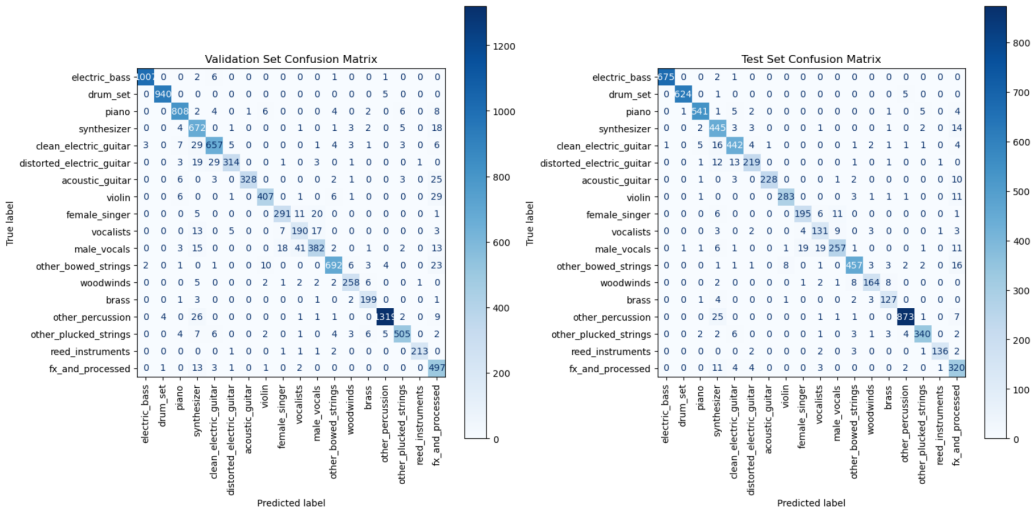


Figure 12: Confusion matrices for stem model 5 (validation and test sets)

As shown in Figures 11 and 12, the model achieves a validation accuracy around 94% with very stable training. The classification report confirms strong performance, especially for instruments such as `drum_set`, `electric_bass`, and `other_percussion`. Compared to its CQT-based counterpart (Model 4), Model 5 shows improved performance for more nuanced classes such as `male_vocals`, `fx_and_processed`, and `vocalists`, thanks to the mel-scale's better resolution in low and mid frequencies.

3.5.2 Mix – Model 3, with Mel-Spectrogram

For mix data, we applied the same strategy, using the architecture from Mix Model 2 and replacing the CQT with the mel-spectrogram. The model was trained using binary cross-entropy loss for multilabel classification.

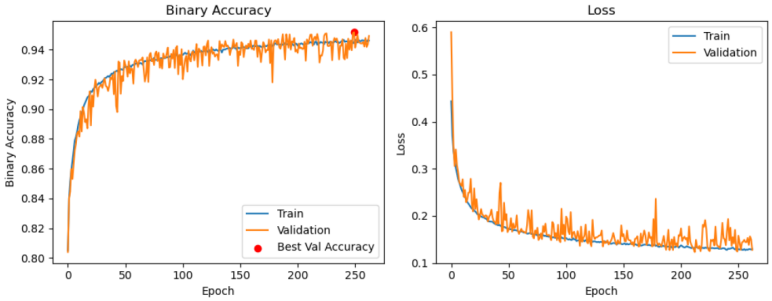


Figure 13: Binary accuracy and loss curves for mix model 3 (mel-spectrogram)

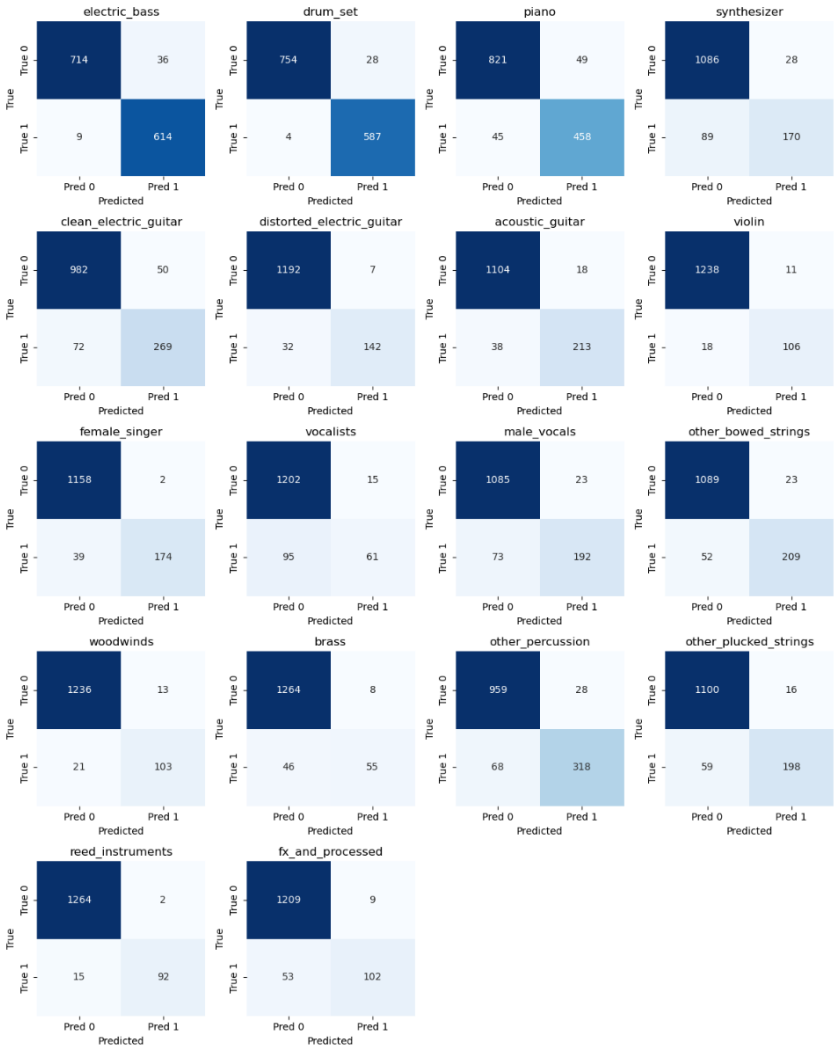


Figure 14: Binary confusion matrices for mix model 3

The training history in Figure 13 shows smoother and slightly better generalization than in the previous CQT-based mix model. The confusion matrices in Figure 14 also indicate better recall for key classes, especially when the instrument is active (`True = 1`), which is critical in multilabel settings. For instance, the recall for classes such as `fx_and_processed`, `reed_instruments`, and `vocalists` is visibly improved.

The classification report supports these observations, with micro and macro F1-scores increasing to 0.87 and 0.84 respectively, compared to 0.84 and 0.79 for the CQT model.

3.5.3 Mel vs. CQT: A Comparison

By comparing Stem Model 4 (CQT) and Model 5 (Mel), as well as Mix Model 2 (CQT) and Model 3 (Mel), we can observe that mel-spectrograms tend to yield better overall performance in our instrument classification task. This is likely due to the mel-scale's closer alignment with the human auditory system, which helps capturing perceptually relevant timbral cues. In both stem and mix configurations, mel-based models achieved higher recall and F1-scores, especially for mid- and low-frequency instruments.

These findings suggest that mel-spectrograms are a more effective representation for our classification task, especially when the goal is to identify when an instrument is playing in polyphonic contexts.

3.6 Hybrid Model: Pretraining on Stem, Finetuning on Mix

To explore a strategy that leverages the advantages of both stem and mix datasets, we introduced a hybrid approach. The idea is to pretrain the model on the large and clean stem dataset, and then adapt it for multilabel mix classification. We started from the same architecture used in the fifth stem model, which had been trained on mel-spectrograms, and replaced the final softmax layer with a sigmoid layer to handle the multilabel nature of mix audio. This modification allows the model to produce independent probability scores for each instrument class, rather than forcing a single prediction.

Furthermore, the normalization of the input data was based on the mix dataset rather than the stem dataset. Since mixture segments contain full polyphonic audio with a higher dynamic range compared to isolated stems, this normalization choice helps the model generalize better to real-world audio mixtures.

Although the model had already been pretrained on stem data, the final training phase was entirely performed on the mix dataset using binary crossentropy loss. As a result, the hybrid model outputs predictions in the same format as a standard mix model, effectively behaving like one, while benefiting from the representational prior learned during pretraining.

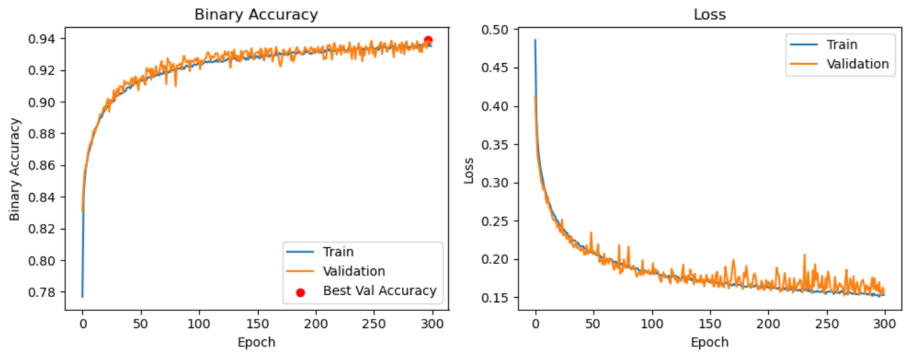


Figure 15: Accuracy and loss curves for the hybrid model when training with mix samples

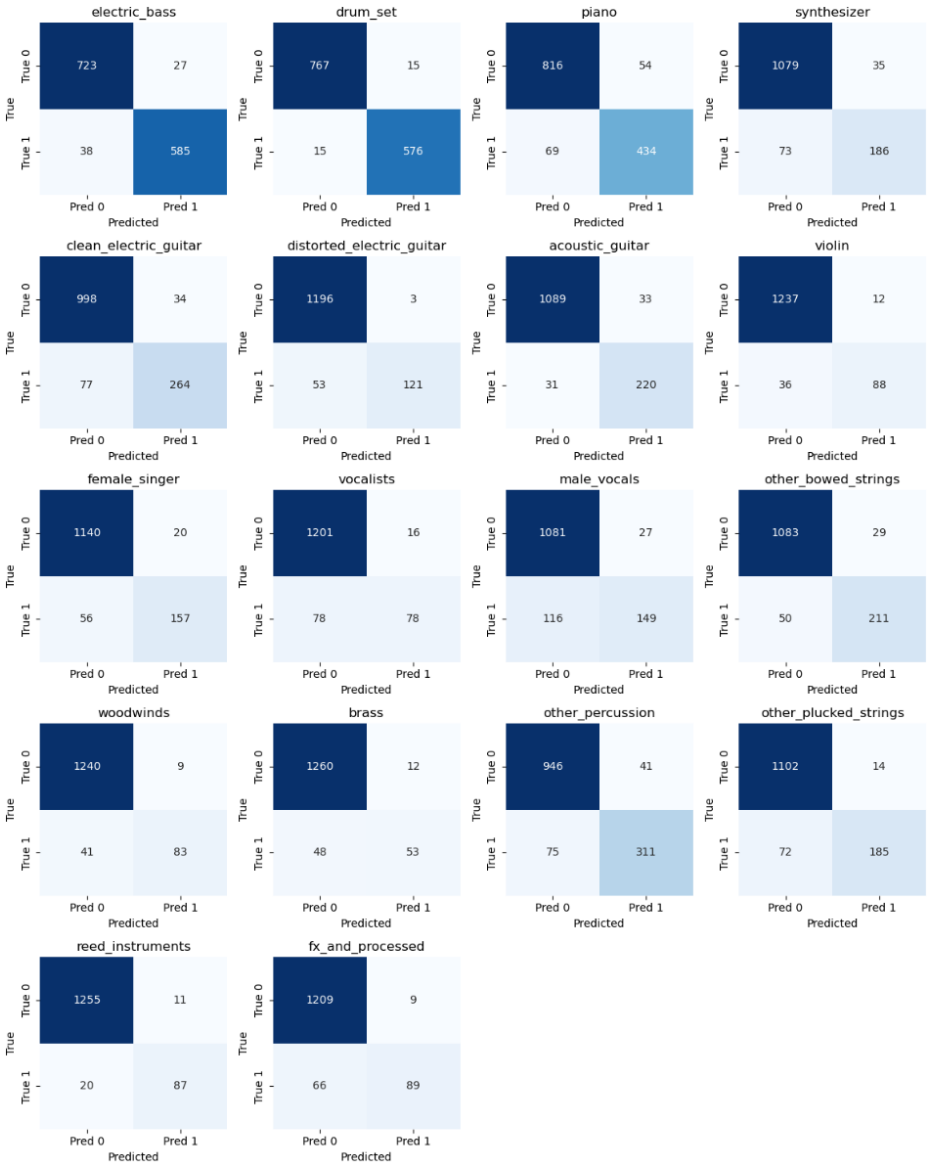


Figure 16: Confusion matrices (validation) for the hybrid model.

The training and validation curves show good convergence, though the binary accuracy saturates slightly below the values achieved by the full mix model. As shown in the confusion matrices, most instrument classes are detected reasonably well, although some classes like `vocalists`, `fx_and_processed`, and `male_vocals` exhibit a drop in recall, indicating difficulty in detecting true positives in polyphonic contexts.

The classification report reflects these observations, with an average sample-wise F1-score of 0.81 on the test set. Precision remains high across most classes, but recall drops in several cases, suggesting the model may be conservative in assigning labels. Still, the hybrid model offers a good compromise between performance and training efficiency, especially in low-resource or transfer-learning scenarios.

3.6.1 Comparison with Mix Model 3

When compared to the third mix model (trained directly on mix data using mel-spectrograms), the hybrid model performs slightly worse across most metrics. This is expected, as the mix model is optimized end-to-end for multilabel classification, whereas the hybrid model is constrained by its pretraining, even though we thought this could be beneficial.

3.7 Reduced Class Models: Model 6 for Stem and Model 4 for Mix

To further simplify and improve classification performance, we explored a class reduction strategy applied to both stem and mix datasets. The motivation is twofold:

- **Merging vocal categories**: `female_singer`, `vocalists`, and `male_vocals` were merged into a single class called `voices`. These classes often share similar spectral features and were frequently confused in previous models, justifying the merge.
- **Class removal**: `synthesizer` and `fx_and_processed` were removed. These categories showed both lower occurrence and poor classification performance across previous experiments, indicating a weaker representation in the dataset and less consistent labeling.

Figures 17 and 18 illustrate the updated class distribution after reduction.

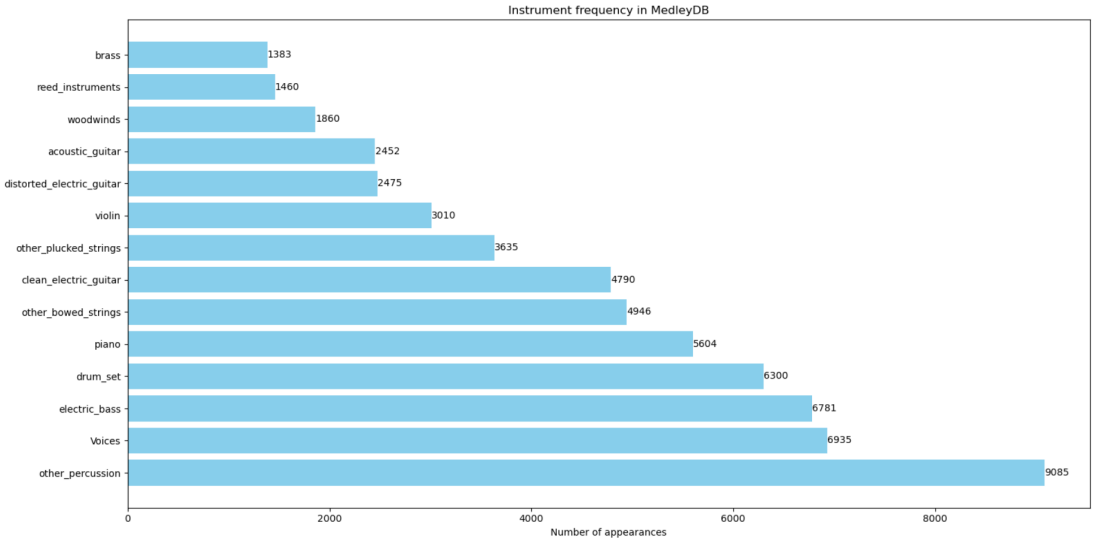


Figure 17: Updated class distribution for Stem dataset

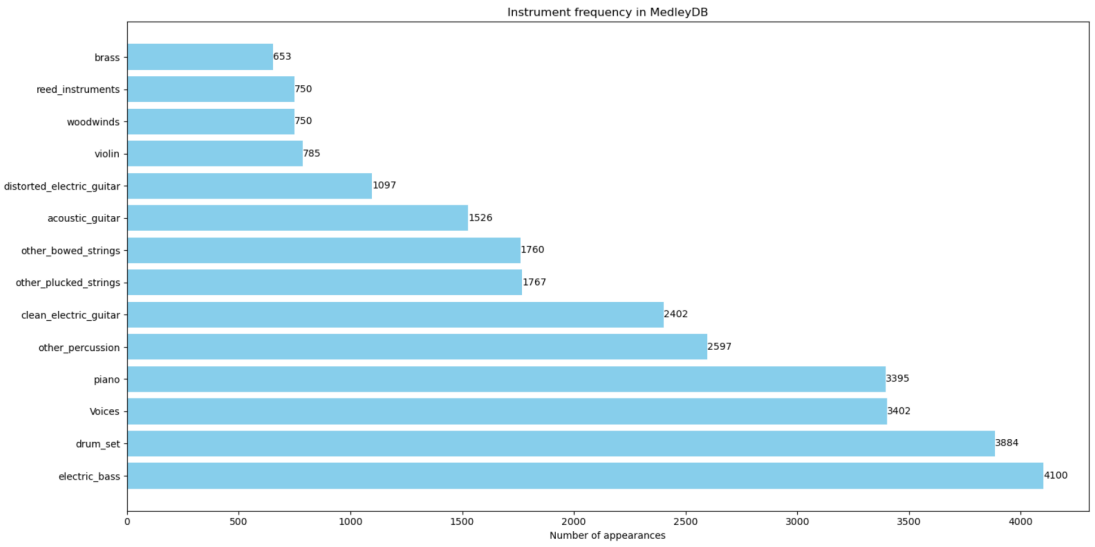


Figure 18: Updated class distribution for Mix dataset

3.7.1 Model 6 - Stem with Reduced Classes

Model 6 maintains the architecture of the previous best-performing CNN using mel-spectrogram input. It was retrained on the reduced stem dataset. The accuracy and loss curves are shown in Figure 19, and the confusion matrix in Figure 20.

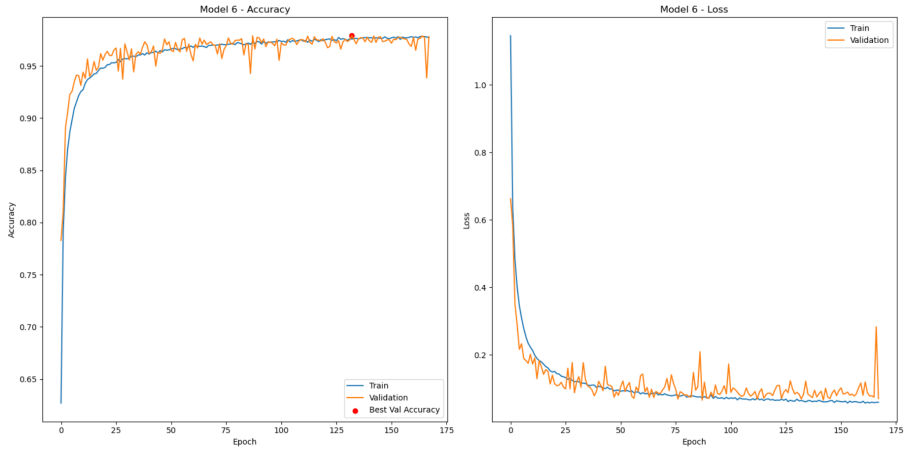


Figure 19: Training and validation accuracy/loss for Model 6 (Stem, reduced classes)

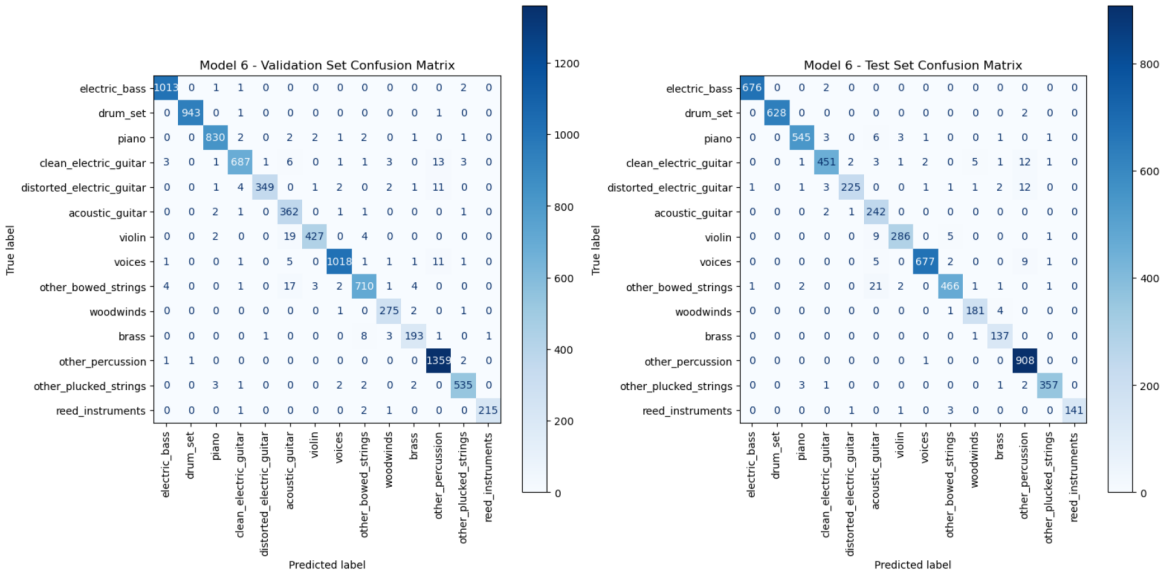


Figure 20: Confusion matrix for validation and test set of Model 6 (Stem)

Model 6 achieves excellent accuracy across both validation and test sets, with an overall test accuracy of 97%. Most classes reach an F1-score above 0.95, showing that reducing class granularity significantly improved consistency and precision.

3.7.2 Model 4 - Mix with Reduced Classes

We also applied the same class reduction to the mix data and trained Model 4 using mel-spectrograms. The accuracy/loss curves and confusion matrix are provided in Figures 21 and 22.

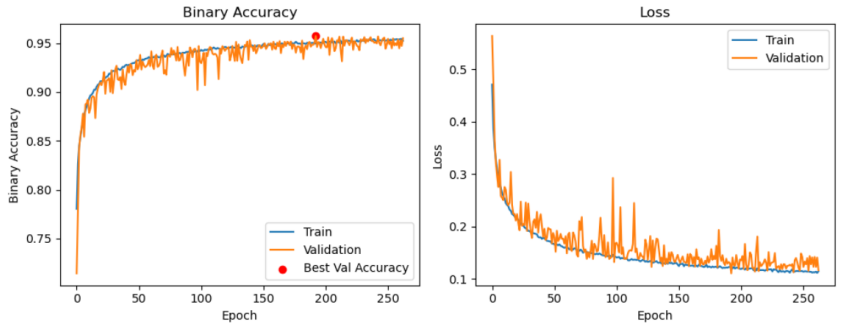


Figure 21: Training and validation binary accuracy/loss for Model 4 (Mix, reduced classes)

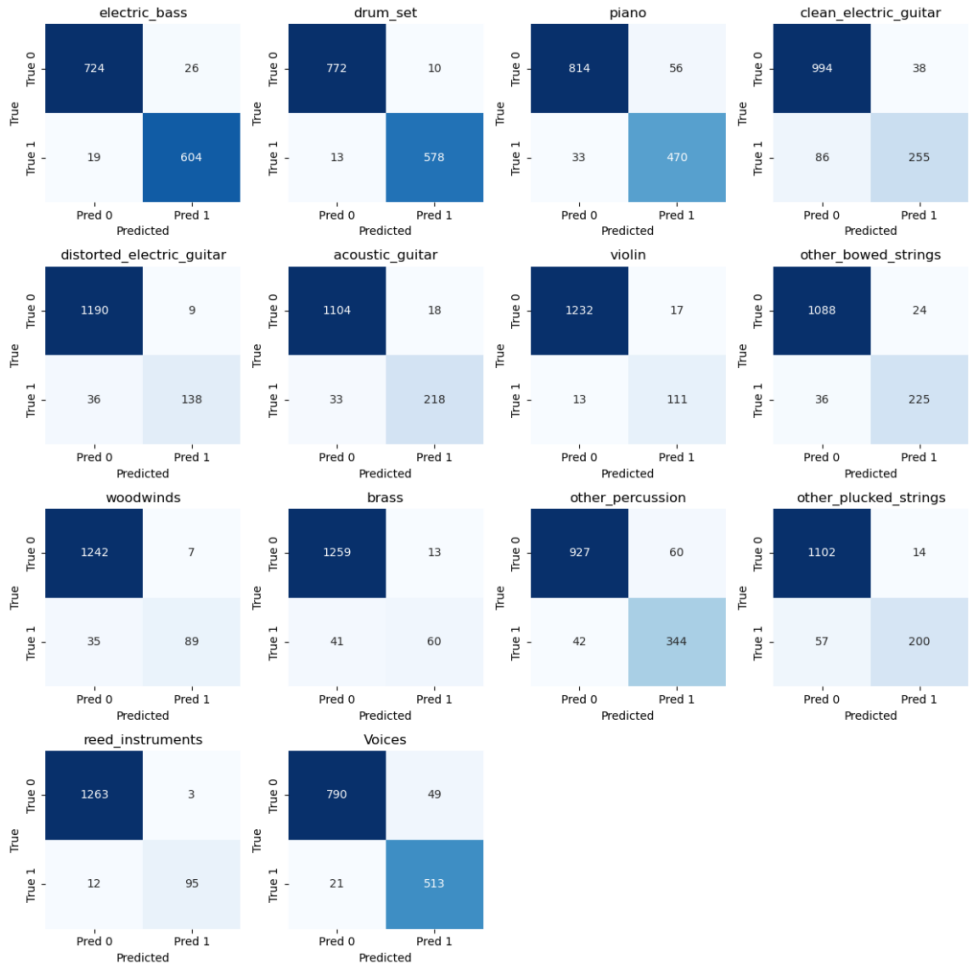


Figure 22: Per-class binary confusion matrices for Model 4 (Mix)

Model 4 also shows solid performance with an overall test accuracy of 89% and macro F1-score of 0.88. The merged `voices` class benefits from consolidation, while rare and previously confusing categories like `synthesizer` are no longer a source of ambiguity. Notably, per-class recall is consistently high across most instruments, which is crucial for correct detection of instrument presence in a mix.

3.7.3 Comparison with Previous Models Before Class Reduction

By reducing the number of classes, both models benefit from a more balanced and semantically meaningful class structure. The stem model (Model 6) improved its macro-averaged F1-score from 0.92 to 0.97 and reached an overall accuracy of 98%, compared to 94% in the previous setting. Similarly, the mix model (Model 4) improved from a macro F1-score of 0.84 to 0.88 and an overall micro-averaged F1-score of 90%, indicating a clear gain in classification quality.

Furthermore, this reduction led to sharper and less ambiguous confusion matrices, with most instruments achieving high true positive rates and minimal confusion. The model was thus able to allocate more discriminative power to a reduced, more stable set of targets.

In conclusion, class reduction not only simplified the classification task but also enhanced model performance and interpretability, especially for complex or overlapping classes like vocals and effects.

3.8 Conclusion of the models

The goal of this project was to build robust models for musical instrument recognition in both monophonic (stem) and polyphonic (mix) contexts, in order to have a clear guideline when developing further models. Through careful dataset curation, feature engineering, and systematic model comparison, we have drawn several key conclusions.

1. Impact of class reduction

One of the most effective strategies was the reduction and regrouping of the original 82 instrument labels into 18, and later fewer, semantically coherent classes. This significantly improved both the classification accuracy and interpretability, particularly for ambiguous or underrepresented classes. The stem model (Model 6) with reduced classes achieved up to **97% accuracy** and a macro F1-score of **0.97**, while the corresponding mix model (Model 4) reached a macro F1 of **0.88**.

These results confirm that class reduction is a powerful tool to enhance model reliability and robustness, or in other terms, that depending on our requirements and available resources, the selection of the proper classes is crucial to obtain good results.

2. Comparison of spectral representations

Across multiple experiments, the use of mel-spectrograms consistently outperformed Constant-Q Transform (CQT) representations. Mel features led to better recall and F1-scores, especially for mid- and low-frequency instruments, and proved more effective in detecting nuanced timbral differences in overlapping signals. For instance, Mix Model 3 (Mel) outperformed its CQT counterpart (Mix Model 2) with a macro F1 increase from **0.79 to 0.84**.

3. Model complexity vs performance

While the most complex models (e.g., Model 1 for stems, Model 1 for mixes) achieved the highest absolute accuracy, they came at the cost of longer training times and more parameters. Lightweight models such as Stem Model 4 and Mix Model 2 achieved competitive performance with far fewer resources, making them suitable for deployment in real-time or resource-constrained environments. The trade-off between accuracy and efficiency must thus be considered according to the application's needs.

4. Polyphonic detection is inherently harder

As expected, classification in polyphonic mixtures proved more challenging than in isolated stems. This is likely due to the increased acoustic complexity and label ambiguity in mixes, where instruments may overlap or be masked. Even so, the multilabel CNNs demonstrated strong performance, with micro F1-scores around **0.90** for the best mix models. The slightly lower performance of the hybrid model further suggests that direct optimization on mix data leads to better multilabel generalization.

Overall, this project demonstrates the feasibility and effectiveness of convolutional models for musical instrument classification, especially when guided by appropriate feature selection, class simplification, and balanced training strategies.

4 Future directions

Further improvements could be achieved by incorporating attention mechanisms or transformers, data augmentation techniques, better balance between classes, or using pretrained embeddings. Additionally, future work may explore hierarchical classification strategies or few-shot learning to better address rare instrument classes.

In the next points, we present two suggested ways to help the model for mix to better perform when labeling the samples.

4.1 Instrument-Genre Relationship Analysis

Firstly, we conducted an exploratory analysis of the relationship between instrumentation and musical genre. Since the MedleyDB metadata includes both stem-level instrument annotations and genre labels for each track, we were able to statistically investigate how frequently different instruments occur across musical genres.

To do so, we parsed all available YAML metadata files in the dataset and, for each track, extracted both its annotated genre and the list of unique instruments involved in the stems. After cleaning and standardizing instrument names, we built a binary matrix encoding the presence of each instrument per track. We then aggregated this data by genre, resulting in a large instrument $\times$ genre matrix where each cell counts the number of times an instrument was observed within a particular genre.

To highlight patterns and normalize for genre frequency, we normalized for each genre the matrix by dividing each cell by the total number of instrument annotations in that genre. The final result is a heatmap that visually represents the relative frequency of each instrument across genres:

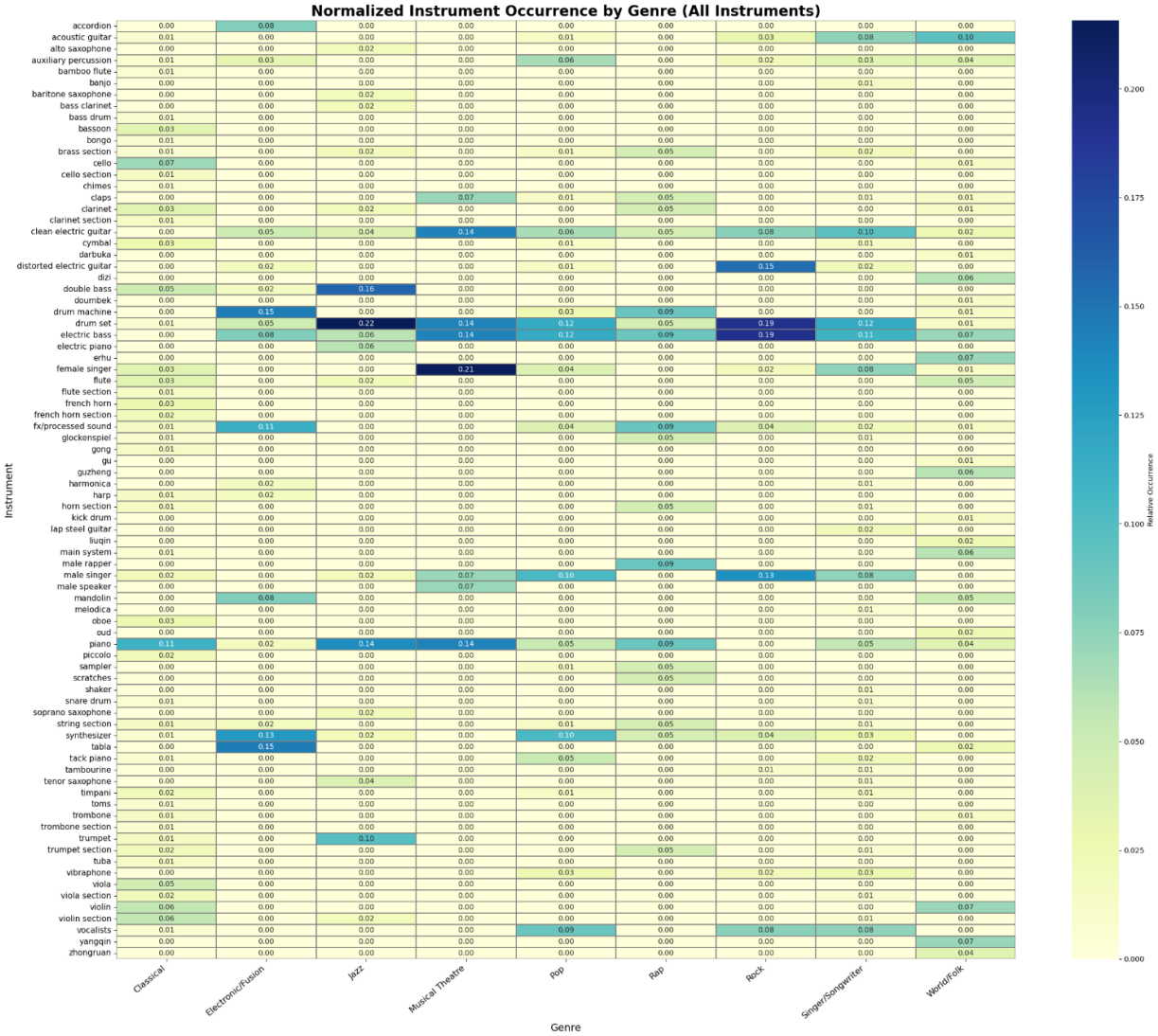


Figure 23: Normalized instrument occurrence by genre in the MedleyDB dataset. Values are row-normalized within each genre

From the heatmap, we can observe strong genre-instrument associations. For instance, distorted electric guitars are predominant in rock-related genres, while clean electric guitars appear more often in pop or singer-songwriter tracks. Voices and electric bass are common across nearly all genres, while brass, woodwinds, and certain classical instruments show more genre-specific distributions.

This visualization supports the potential of using genre as contextual information in future models, either for genre-aware classification or for guiding regularization strategies based on expected instrument co-occurrence patterns.

4.2 Instrument Co-occurrence Analysis

Another relevant exploratory direction we pursued involved the analysis of **instrument co-occurrence patterns** in the MedleyDB dataset. The objective here was to investigate which instruments tend to appear together within the same musical excerpts, thereby modeling typical instrument combinations across tracks.

To compute the co-occurrence matrix, we first parsed all the stem-level metadata to extract, for each track, the list of instruments involved. We then iterated over each track and, for every unordered pair of instruments present, we increased the corresponding counter in a square matrix of shape $N \times N$, where N is the number of unique instruments across the dataset, and normalized over the amount of tracks the instrument it's present.

This means the matrix it's not symmetric anymore, and should be read by rows (the direction of normalization), so for one cell, the vertical axis is the one telling us in respect to which instrument it's done the normalization. The process effectively captured how many times each instrument pair co-occurred throughout the dataset. Notice that we didn't consider the appearances of an instrument with itself.

The resulting co-occurrence matrix, visualized as a heatmap in Figure 24, uses a perceptually uniform `magma` colormap and displays the exact number of co-occurrences between instrument pairs.

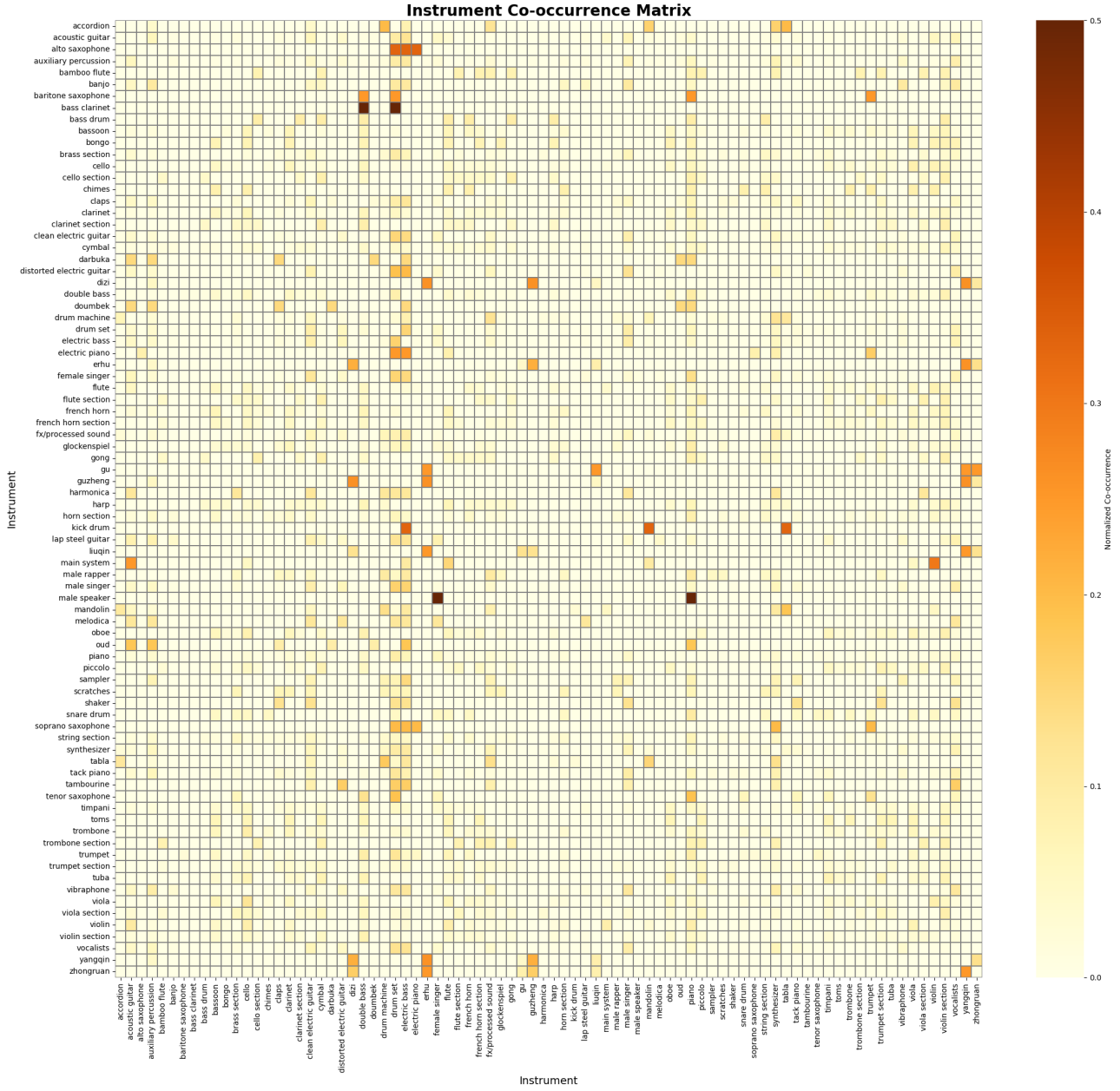


Figure 24: Filtered instrument co-occurrence matrix across MedleyDB tracks. Higher intensity indicates more frequent co-occurrence between the corresponding instrument pairs.

This matrix is particularly useful for understanding *typical orchestration strategies* and *common ensemble structures*. For instance, it highlights strong co-occurrence between instrument pairs such as `male speaker` and `female singer`, or between `electric piano` and instruments like `electric bass` or `drum set`. These relationships may provide valuable prior knowledge in the context of multi-instrument recognition, because when seeing instruments like `bass clarinet`, the model can expect `double bass` to be also present, but not a `french horn`.

5 References

- V. N. Kumar. *Predominant Musical Instrument Classification based on Spectral Features.* GitHub repository. Available at: `https://github.com/vntkumar8/musical-instrument-classification`
- V. Lostanlen and C.-E. Cella. *Deep Convolutional Networks on the Pitch Spiral for Musical Instrument Recognition.* In: *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2016. GitHub repository: `https://github.com/lostanlen/ismir2016`
- R. M. Bittner, J. Wilkins, H. Yip, and J. P. Bello. *MedleyDB 2.0: New Data and a System for Sustainable Data Collection.*